

CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY (CSPIT)

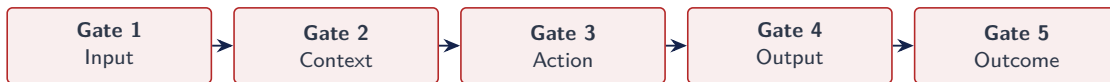
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

PROJECT MASTER TECHNICAL REPORT

MIRAGE 3.0: AI Execution Assurance Platform

A Unified Architecture for Governed AI Transactions, Capability Enforcement, and Epistemic Reality Verification



Project Metadata and Student Investigators

Student Investigator 1:	Vedant Panchal (ID: 23AIML042)
Student Investigator 2:	Dax Virani (ID: 23AIML076)
Degree Program:	Bachelor of Technology in Artificial Intelligence & Machine Learning
Department / College:	Dept. of AI & ML, CSPIT, CHARUSAT
Academic Year:	2026–2027
Project Category:	Enterprise AI Safety, Autonomous Execution Assurance, Robust Systems
Engineering Status:	Perimeter security hardened (all three P0 vulnerabilities closed). All five assurance gates (Gates 1–5) fully implemented and hardened. Epistemic reality verification with monotonic state machine operational. 511 automated regression tests passing (511 passed, 0 failures). 9-scenario live demonstration validated (LIVE_DEMO_READY). Submitted for faculty review and approval.

Abstract

As AI systems evolve from conversational assistants into autonomous agents that call tools, query databases and trigger consequential actions, hallucination detection alone no longer bounds enterprise risk. An agent that can write to a database, issue a payment or exfiltrate private context fails in ways that a text-level check cannot see.

MIRAGE 3.0 is an *AI Execution Assurance Platform*: a mediation fabric between AI systems and the external world, built on the promise “*Control AI before it acts. Verify AI after it acts.*” Every agent interaction is wrapped in a governed **AI Transaction** and evaluated across **five assurance gates**: (1) Input Assurance (identity, capabilities, prompt-injection defence, DLP); (2) Context Assurance (retrieval provenance, memory integrity, information-flow control with taint tracking); (3) Action Assurance (GovernedToolProxy, cryptographic action contracts, single-use context-bound approval tickets, sliding-window blast-radius limits); (4) Output Assurance (atomic claim decomposition; fusion of Qdrant retrieval, Groq self-consistency sampling and DeBERTa-v3 NLI; split-conformal calibration at $1 - \alpha = 0.95$; LangGraph self-healing; completion invariants); and (5) Outcome Assurance (Epistemic Reality Verification across four observation classes, seven canonical outcome statuses, monotonic state transitions, and Output $\leftrightarrow$ Outcome reconciliation).

Across Phases 0 through 5.5, the platform underwent rigorous red-team hardening, closing all identity-boundary, SSRF redirect, and SQL injection vectors. This report presents the authoritative architecture, mathematical formulations, live telemetry from Groq/Qdrant/PostgreSQL execution, 511 passing automated regression tests, a competitive landscape, and an audited implementation inventory for faculty review.

Reading guide: what is built and what is specified

Throughout this report, components carry one of four labels, summarised in Table 9 and Figure 19: **Implemented** (running and tested), **Partial** (scaffolding exists, integration under way), **Specification** (designed and documented, not yet coded), and **Roadmap** (scheduled work). All five assurance gates (Gates 1–5), the GovernedToolProxy, the AI Transaction state machine, and the Reality Verifier are fully **Implemented** and validated by 511 automated tests.

Keywords: AI execution assurance, governed AI transactions, epistemic reality verification, assurance gates, split-conformal prediction, information-flow control, LangGraph self-correction, multi-tenant security.

Contents

1	Executive Summary and Strategic Reset	1
1.1	From hallucination detector to assurance fabric	1
2	Problem Statement: Fourteen AI Execution Failure Modes	2
2.1	From chat verification to action assurance	2
2.2	Taxonomy of failure modes	2
2.3	Formal problem formulation	3
3	Plain-English Analogy	4
4	Product Taxonomy: Six Integration Surfaces	4
5	System Architecture: Tri-Plane Separation with Observability	5
6	The Five Assurance Gates	7
6.1	Gate 1 – Input Assurance	7
6.2	Gate 2 – Context Assurance	7
6.3	Gate 3 – Action Assurance	7
6.4	Gate 4 – Output Assurance	8
6.5	Gate 5 – Outcome Assurance	8
6.6	Message-level view of one governed transaction	8
7	Governance Primitives and the AI Transaction	10
7.1	Authoritative entities	10
7.2	Bounded iterative ReAct loop	10
7.3	The AI Action Contract	11
8	Gate 5: Epistemic Reality Verification	11
8.1	Four observation classes and seven canonical outcome statuses	11
8.2	Production verifier adapters and monotonic state transitions	11
8.3	Canonical cryptographic fingerprinting: <code>mirage.outcome.v1</code>	12
8.4	Output $\leftrightarrow$ Outcome consistency reconciliation	12
9	Security Architecture and Threat Model	13
9.1	Resolved P0 identity-boundary vulnerabilities	13
9.2	Dynamic information-flow control	14
9.3	Sliding-window blast radius	14
9.4	Perimeter hardening: GovernedToolProxy, anti-SSRF, and single-use approvals	15
9.5	Tamper-evident audit ledger	15
10	Gate 4: Output Assurance and the Verification Engine	16
10.1	Evidence signals	16

10.2 HRS fusion and the dominance rule	16
10.3 Split-conformal calibration and attribution	17
10.4 LangGraph closed-loop self-healing	17
11 Validated Empirical Telemetry	18
11.1 Runtime model identity probes	18
11.2 Grounded versus hallucinated verification	18
11.3 Persistence and fault-injection probes	19
11.4 Automated test regression suite telemetry	19
12 Cost Optimisation: The Verification Escalation Ladder	20
13 Comprehensive Competitive Landscape	20
13.1 Architectural differentiation	22
14 Implementation Status Inventory	22
15 Live Demonstration Walkthrough	23
16 Research Innovations versus Inherited Heritage	24
17 Strengths, Limitations and Threats to Validity	24
17.1 Strengths	24
17.2 Limitations	25
17.3 Threats to validity	25
18 Phased Roadmap and Future Work	25
19 Conclusion and Faculty Approval Request	26

List of Figures

1	Scope expansion from MIRAGE 2.x (Gate 4 only) to MIRAGE 3.0 (five gates).	1
2	Traceability matrix from failure modes to assurance gates.	3
3	Six integration surfaces converge on one assurance fabric between clients and the real world.	5
4	Level-1 data flow across the core subsystems.	5
5	Tri-plane architecture with a cross-cutting observability plane.	6
6	End-to-end five-gate pipeline with remediation paths (red) and environment interaction.	8
7	Sequence of a governed AI transaction (solid: request or command; dashed: return).	9
8	Target AI Transaction state machine (versioned, with the governed ReAct loop).	10
9	Gate 5 decision tree mapping probe outcomes to the seven epistemic states (green: consistent, amber: unverifiable, red: discrepancy requiring action).	13
10	Perimeter defence in depth: header stripping, database-resolved identity and row-level security.	14
11	Security lattice and taint propagation: the join of two labels governs egress.	14
12	Illustrative blast-radius dynamics ($\lambda = 0.35$, $\omega = 1$, impacts 1.0, 1.2, 0.8, 1.4): the fourth action would breach $\mathcal{B}_{\max}$ and is denied.	15
13	SHA-256 hash-chained audit ledger: intact versus tampered chain.	16
14	Gate 4 verification engine: from claim decomposition to emission, self-healing or escalation.	17
15	LangGraph closed-loop self-correction with a mandatory re-verification gate.	18
16	HRS for the two live scenarios with 95% conformal intervals ($\hat{q} = 0.0650$). The rewrite drops scenario B from 0.85 to 0.044, below the 0.30 re-verification gate.	18
17	Verification escalation ladder: cost and latency grow with each rung.	20
18	Lifecycle coverage of compared systems, derived from the “assurance scope” and “action governance” rows.	21
19	Implementation status by gate and platform component: all five assurance gates fully implemented and validated (11 implemented, 1 roadmap item).	23
20	Phased engineering and research roadmap: core five-gate architecture completed; empirical corpora certification scheduled.	25

List of Tables

1	The fourteen failure modes addressed by MIRAGE 3.0.	2
2	MIRAGE 3.0 product interfaces.	4
3	The seven canonical outcome statuses of Gate 5 Reality Verification.	12
4	Remediated P0 vulnerabilities.	13
5	Multi-signal evidence matrix of the Gate 4 verification engine.	16
6	Live telemetry captured on two production scenarios.	19
7	Architectural comparison, part A: academic baselines and guardrail libraries.	20
8	Architectural comparison, part B: commercial evaluation and cloud platforms.	21
9	Implementation status inventory across platform subsystems.	22
10	Inherited heritage versus novel contributions.	24

1 Executive Summary and Strategic Reset

1.1 From hallucination detector to assurance fabric

MIRAGE 2.x was a hallucination detector: it decomposed model completions into claims and scored factual consistency using retrieval, sampling and entailment signals. That scope is scientifically sound but no longer sufficient. Production agents now use tool protocols (Model Context Protocol, function calling), long-term vector memory, code interpreters and internal APIs. In that setting:

- a factually accurate reply can still carry out an unauthorised database drop;
- a polite completion can be the by-product of an indirect prompt injection that leaked tenant secrets;
- an agent can claim an API call succeeded when it timed out, or claim failure when it partly mutated production state.

The MIRAGE 3.0 product statement

MIRAGE is not merely a hallucination detector, gateway, observability dashboard or guardrail wrapper. It is an **AI Execution Assurance Platform**.

“MIRAGE sits between AI systems and the real world, inspecting inputs, context, actions, outputs and outcomes to enforce identity, security, policy, reliability and cost controls, while using adaptive verification and model routing to reach the required level of assurance at the lowest practical cost.”

Operational promise: *Control AI before it acts. Verify AI after it acts.*

All scientific assets of MIRAGE 2.x are preserved: retrieval-augmented verification (RAV), self-consistency sampling (SCS), natural language inference (NLI), visual grounding (VGS), the Hallucination Risk Score (HRS), isotonic and Mondrian split-conformal calibration [3, 4], TreeSHAP attribution [7], and LangGraph self-healing. They are repositioned as the *Verification Engine* inside Gate 4 (Figure 1).

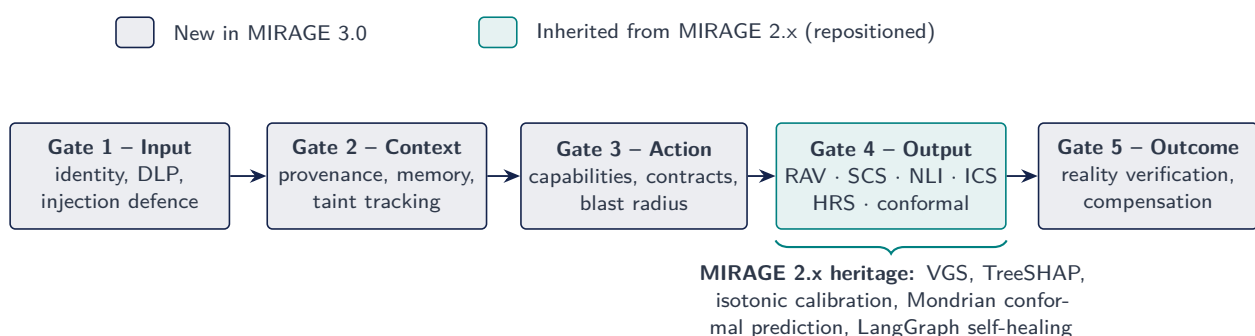


Figure 1: Scope expansion from MIRAGE 2.x (Gate 4 only) to MIRAGE 3.0 (five gates).

2 Problem Statement: Fourteen AI Execution Failure Modes

2.1 From chat verification to action assurance

Between 2022 and 2024 the central enterprise question was “*Was the answer factually correct?*” For autonomous agents the question becomes: *Was the agent permitted to take that action, did it stay inside its capability boundary, did the action really happen, and is production left in a consistent state?*

Conventional guardrails (regex matchers, prompt wrappers, LLM-as-judge classifiers) treat interactions as isolated strings. They lack a transaction concept, cannot enforce identity boundaries, cannot track provenance across multi-turn reasoning, and cannot confirm that external side effects occurred.

2.2 Taxonomy of failure modes

#	Failure mode	Mechanism	MIRAGE 3.0 countermeasure
1	Unauthenticated agent identity	Agents call tools with forged or ambient credentials.	Cryptographic JWT/API identity resolution; ASGI header stripping.
2	Direct or indirect prompt injection	Adversarial instructions in queries or external documents override system goals.	Gate 1 deterministic preflight and semantic jailbreak filters.
3	Sensitive data exfiltration (DLP)	PII, credentials or IP leak via completions or tool calls.	Information-flow control with taint tracking; egress policy.
4	Context and retrieval poisoning	Adversarial or stale RAG chunks corrupt reasoning.	Gate 2 provenance, freshness and integrity checks.
5	Excessive agency	Agent attempts operations beyond its charter.	Capability-based security with default-deny governance.
6	Malformed tool parameters	Invalid types, SQL injection, unconstrained file paths.	Gate 3 schema validation and pre-execution invariants.
7	Blast-radius accumulation	Low-risk micro-actions add up to catastrophic damage.	Sliding-window cumulative risk accounting.
8	Extrinsic factual fabrication	Generation invents facts unsupported by references.	Gate 4 RAV over Qdrant.
9	Stochastic semantic dispersion	Epistemic uncertainty yields diverging samples.	Parallel Groq self-consistency sampling.
10	Logical contradiction	Claims contradict authoritative ground truth.	DeBERTa-v3 cross-encoder NLI.
11	Visual / multimodal discrepancy	Text describes objects absent from the image.	OpenCLIP similarity and LLaVA-1.6 consistency.
12	Epistemic reality discrepancy	Agent asserts success but the world is unchanged.	Gate 5 postcondition probes against ground truth.
13	Partial failure without compensation	Multi-step workflow fails midway leaving side effects.	Transaction lifecycle with compensation and rollback.
14	Non-repudiation and audit failure	Cannot prove who authorised what, or on what evidence.	Tamper-evident SHA-256 hash-chained audit ledger.

Table 1: The fourteen failure modes addressed by MIRAGE 3.0.

Figure 2 traces each failure mode to the gate(s) that own it. Every failure mode has at least one

primary owner, and failure mode 14 is cross-cutting through the audit ledger.

	G1 Input	G2 Context	G3 Action	G4 Output	G5 Outcome	Audit ledger
1 Unauthenticated agent identity	●					○
2 Prompt injection (direct / indirect)	●	○				
3 Sensitive data exfiltration	●	●				
4 Context and retrieval poisoning		●				
5 Excessive agency	○		●			
6 Malformed tool parameters			●			
7 Blast-radius accumulation			●			
8 Extrinsic factual fabrication				●		
9 Stochastic semantic dispersion				●		
10 Logical contradiction				●		
11 Visual discrepancy				●		
12 Epistemic reality discrepancy					●	
13 Partial failure, no compensation			○		●	
14 Non-repudiation / audit failure						●

● primary owner ○ supporting control

Figure 2: Traceability matrix from failure modes to assurance gates.

2.3 Formal problem formulation

Formal problem formulation

Let an agent interaction be a sequence of state transitions over an environment $\mathcal{S}$. For an untrusted input $x \in \mathcal{X}$, an assembled context bundle $\mathcal{K}$ and a proposed action sequence $\mathbf{a} = (a_1, \dots, a_T)$, the objective is a mediation fabric $\mathcal{M}$ such that:

1. no action a_t executes unless authorised by the capability assignment $\mathcal{C}(\text{agent}, a_t)$ and the cumulative blast radius in window W stays bounded, $\sum_{\tau \in [t-W, t]} \text{Risk}(a_\tau) \leq \mathcal{B}_{\max}$;
2. output hallucination risk is reported with a split-conformal interval $\mathcal{C}_{1-\alpha}(X)$ that satisfies the marginal coverage property $\mathbb{P}(Y \in \mathcal{C}_{1-\alpha}(X)) \geq 1 - \alpha$;
3. the post-execution state $\mathcal{S}_{t+1}$ is checked against the declared postconditions $\Psi_{\text{post}}(a_t)$, so that discrepancies between what the agent reports and what the environment contains are *detected and classified*, under a target end-to-end latency of $P_{95} < 3.0$ s.

Scope note

Objective 3 is a design target. The P95 latency bound has not yet been measured end-to-end across all five gates; only Gate 4 component latencies are reported (Table 5).

3 Plain-English Analogy

The ultra-fast executive assistant and the corporate assurance desk

Imagine a company hires an extremely fast executive assistant, **Sam**, who stands for any frontier agent. Sam can draft fifty emails, write database scripts and call banking APIs in seconds. Sam also has no common sense about consequences and may book flights on an unauthorised card, send merger papers to a competitor, or write “I wired \$50,000 to Vendor X” when the transfer actually timed out.

MIRAGE 2.x was a proofreader. It sat next to Sam and caught “Paris is in Germany”, but could not stop Sam from pressing *Delete Database*.

MIRAGE 3.0 is a five-desk assurance desk between Sam and company infrastructure:

1. **Security and Reception (Gate 1).** Is the order authentic? Does it hide instructions or contain unredacted Social Security numbers?
2. **Research Archive (Gate 2).** Do reference documents come from trusted safes? Sensitive material is tagged, and tampering is detected.
3. **Comptroller (Gate 3).** Before touching any tool, Sam presents a formal *Action Contract*. Does Sam hold the right badge? Is the spend within today’s limit?
4. **Fact-Checking Bureau (Gate 4).** The draft is split into statements, checked against references, tested for consensus and logic, and rewritten with citations if needed.
5. **Field Inspector (Gate 5).** When Sam says “I updated the record”, the inspector logs in and checks the row. If it is missing, the discrepancy is logged and rollback begins.

4 Product Taxonomy: Six Integration Surfaces

MIRAGE 3.0 exposes six integration surfaces (Table 2), all routed into the same assurance fabric (Figure 3).

Surface	Consumers	Mechanism and contract
1. Drop-in reverse proxy	Enterprise backends, existing LLM apps	OpenAI-compatible <code>/v1/chat/completions</code> . Intercepts the prompt/response lifecycle, applies Gates 1 and 4, requires no client refactoring.
2. Transaction REST API	Agent orchestrators, microservices	POST <code>/v1/transactions</code> and <code>/v1/transactions/{id}/step</code> : optimistic-locking state machine for bounded ReAct turns.
3. Direct verification API	Compliance pipelines, evaluation harnesses	POST <code>/v1/verify</code> : returns atomic claims, per-signal scores, TreeSHAP values and conformal intervals.
4. Streaming WebSocket	Frontend copilots, chat UIs	<code>/v1/stream/transactions</code> and <code>/v1/verify/stream</code> : claim-level telemetry and approvals in real time.
5. Governed MCP server	AI IDEs, Cursor, Claude Desktop	Exposes <code>verify_claims</code> and <code>audit_action</code> and also governs downstream MCP tool calls as an inline proxy.
6. React dashboard	Compliance officers, SecOps, safety engineers	React 18 + Vite console (JavaScript/JSX only): live transactions, PSI drift, TreeSHAP waterfalls, PDF certificates.

Table 2: MIRAGE 3.0 product interfaces.

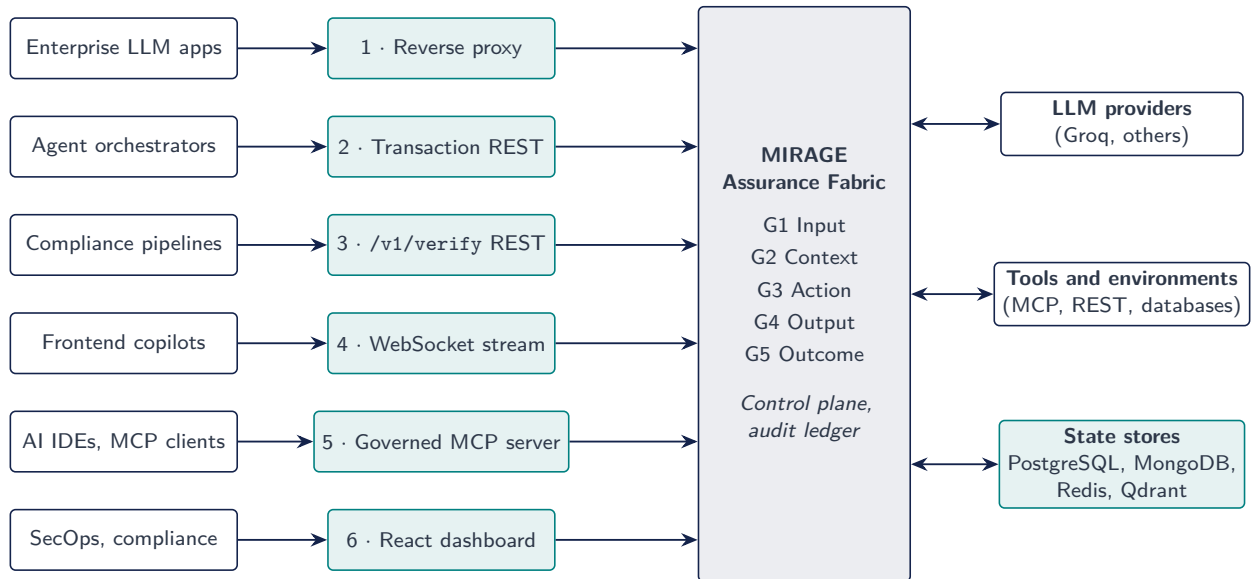


Figure 3: Six integration surfaces converge on one assurance fabric between clients and the real world.

5 System Architecture: Tri-Plane Separation with Observability

MIRAGE separates concerns across three operational planes, reinforced by a cross-cutting observability plane (Figure 5). Figure 4 first shows the Level-1 data flow through the core subsystems.

1. **Control plane (authority).** Canonical definitions of tenants, identities, capabilities, policies, budgets and registered models. Execution nodes cannot invent permissions or override policy.
2. **Execution plane (runtime).** The high-throughput mediation path: FastAPI gateway, intent classifier, risk engine, action governor, verification engine, reality verifier.
3. **Data plane (state).** Partitioned persistence: PostgreSQL 16 (control data and audit ledger under RLS), MongoDB 7 (transaction traces), Redis 7 (tokens, rate limits, working memory), Qdrant (HNSW evidence embeddings [10]).
4. **Observability plane.** OpenTelemetry tracing, Prometheus metrics, structured JSON logs and the SHA-256 hash-chained audit ledger.

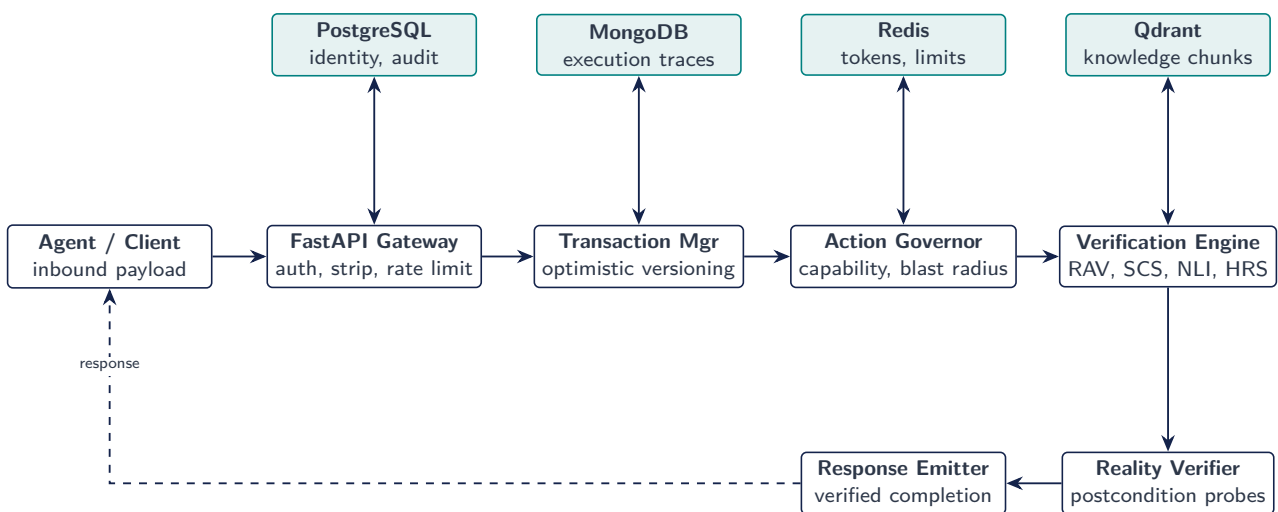


Figure 4: Level-1 data flow across the core subsystems.

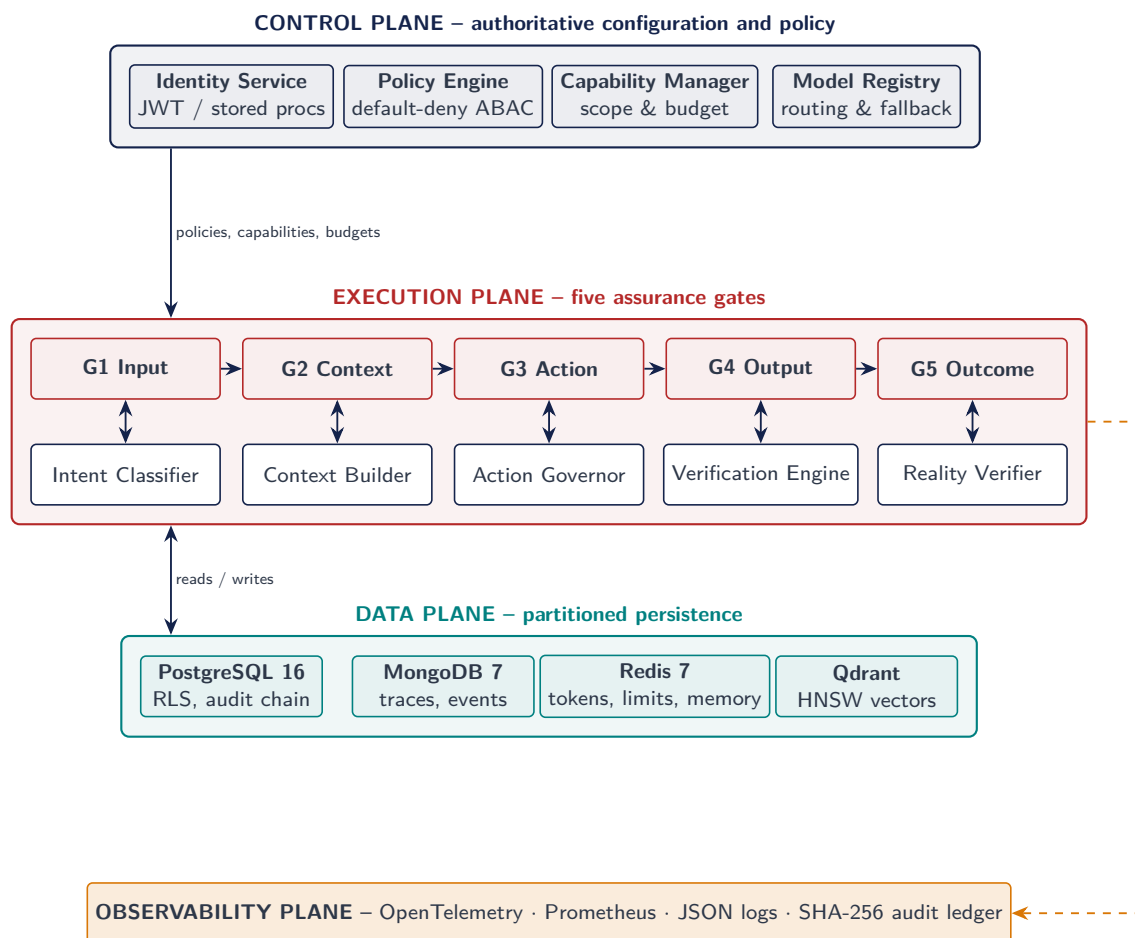


Figure 5: Tri-plane architecture with a cross-cutting observability plane.

6 The Five Assurance Gates

Every interaction passes through an ordered sequence of five assurance gates (Figure 6), each with its own decision space, enforcement boundaries, and failure semantics.

6.1 Gate 1 – Input Assurance

Implementation: `services/input_assurance.py`.

Responsibilities: Verifies caller identity and token authenticity; strips spoofed client headers (X-Role, X-Tenant-ID); classifies inbound intent; detects direct and indirect prompt injection attempts; scans for data loss prevention (DLP) patterns (SSNs, API keys, passwords, bearer tokens); and enforces token-bucket rate limits.

Decisions: ALLOW, TRANSFORM (redact PII/secrets), SANITIZE, ROUTE, REQUIRE_APPROVAL, BLOCK.

Failure mode: Fail-closed; malicious or unauthenticated requests are dropped at the perimeter before any costly LLM inference or tool execution occurs, thwarting denial-of-wallet vectors.

6.2 Gate 2 – Context Assurance

Implementation: `services/context_assurance.py`.

Responsibilities: Audits every context chunk injected into the prompt assembly bundle (RAG retrieved passages, conversation memory, tool results); verifies document provenance, cryptographic hashes, and freshness decay $e^{-\gamma(t-t_0)}$; checks cross-tenant isolation boundaries; detects memory poisoning; and attaches lattice security taint labels (TAINT_PUBLIC, TAINT_INTERNAL, TAINT_CONFIDENTIAL, TAINT_UNTRUSTED).

Decisions: ALLOW, TRANSFORM (redact), QUARANTINE, DROP, BLOCK.

Enforcement: Context chunks carrying untrusted or contradictory provenance are quarantined; working memory dynamically inherits the least upper bound label $\ell_1 \sqcup \ell_2$.

6.3 Gate 3 – Action Assurance

Implementation: `services/action_governor.py` and `services/governed_tool_proxy.py`.

Responsibilities: Acts as the mandatory physical execution boundary between the agent and external tools/APIs. Intercepts proposed actions before invocation; constructs an HMAC-bound **AI Action Contract**; enforces capability-based security; validates arguments against strict JSON schemas; computes sliding-window cumulative blast radius $\mathcal{B}(t)$; inspects outbound targets against SSRF/CIDR blocks and path traversal; and issues single-use, context-bound approval tickets for elevated actions.

Decisions: ALLOW, REQUIRE_HUMAN_REVIEW, DENY.

Enforcement: Default-deny [13]; direct tool bypass is physically impossible because credentials reside exclusively within the proxy. Self-approval by agents is strictly prohibited.

6.4 Gate 4 – Output Assurance

Implementation: `services/output_assurance.py` and `hrs_engine/`.

Responsibilities: Intercepts generated candidate completions before release; splits text into atomic propositions; evaluates multi-signal evidence: dense retrieval (RAV via Qdrant), semantic consistency sampling (SCS via parallel Groq calls), DeBERTa-v3 cross-encoder entailment (NLI), and internal paragraph consistency (ICS); calculates the calibrated Hallucination Risk Score (HRS); computes Mondrian split-conformal coverage intervals at $1 - \alpha = 0.95$; performs TreeSHAP attribution; applies DLP red-team sanitization; triggers LangGraph self-healing rewrites if $HRS > 0.60$; and enforces transaction completion invariants.

Decisions: ALLOW, REWRITE (LangGraph closed loop), BLOCK, REQUIRE_HUMAN_REVIEW.

6.5 Gate 5 – Outcome Assurance

Implementation: `services/reality_verifier.py`.

Responsibilities: Governs post-execution state verification across the external environment; measures actual system state changes rather than relying on agent self-reports; classifies evidence across four observation classes (OBS_DIRECT, OBS_EVENTUAL, OBS_INFERRED, OBS_BLIND); categorizes reality state into seven canonical statuses; executes verifier adapters (Database, REST/API, Celery Worker, File/Log); enforces monotonic state transitions; generates canonical SHA-256 fingerprint records under `mirage.outcome.v1`; and executes Output $\leftrightarrow$ Outcome consistency reconciliation (`reconcile_output_with_outcome`) to prevent phantom successes from misleading users.

Decisions: CONFIRMED, EVENTUAL_PENDING, UNVERIFIED_ACK, DISCREPANT_FAILED, COMPENSATE, ALERT.

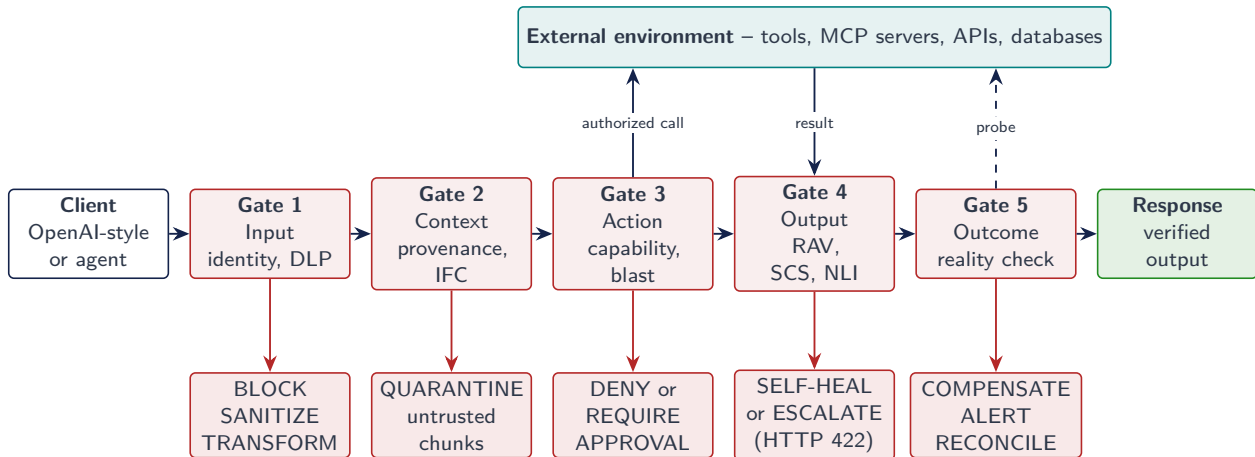


Figure 6: End-to-end five-gate pipeline with remediation paths (red) and environment interaction.

6.6 Message-level view of one governed transaction

Figure 7 shows the message order for a request that needs one tool call, from submission to audited response.

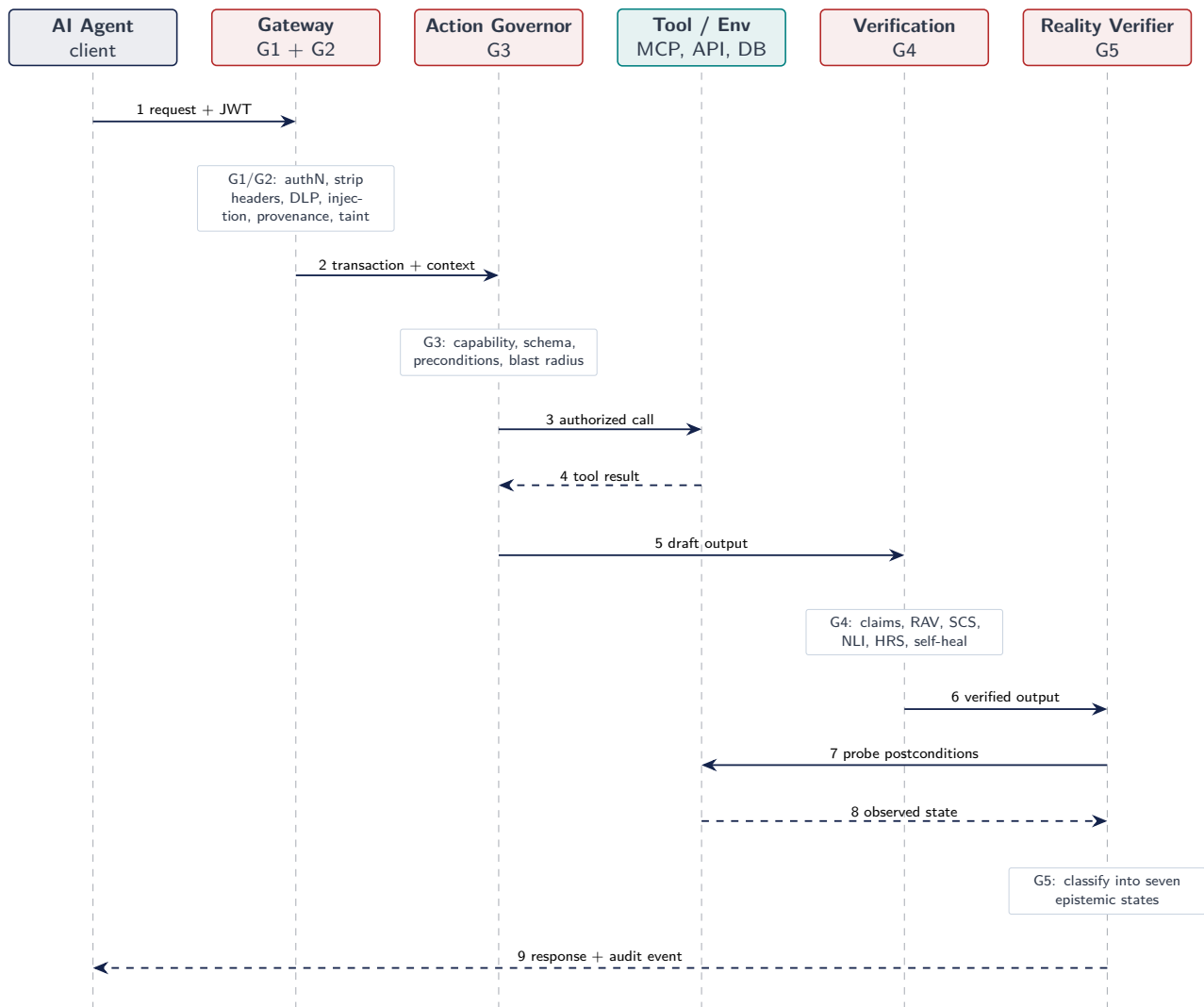


Figure 7: Sequence of a governed AI transaction (solid: request or command; dashed: return).

7 Governance Primitives and the AI Transaction

7.1 Authoritative entities

MIRAGE formalises eleven entities: **Tenant** (isolation boundary enforced by PostgreSQL RLS); **Identity** (authenticated principal bound to a token); **Agent** (autonomous actor under a charter); **Capability** (scoped permission for an action class); **Tool** (REST endpoint, database, MCP server); **Policy** (default-deny declarative rule); **Intent** (classified purpose of a request); **AI Transaction** (stateful unit of execution); **AI Action Contract** (pre-execution declaration); **Risk Assessment** (composite of blast radius, sensitivity and reversibility); and **Audit Event** (immutable ledger entry).

7.2 Bounded iterative ReAct loop

Unlike a stateless proxy, MIRAGE wraps multi-turn Reason–Act loops [11] in an **AI Transaction**. Unbounded agents can loop forever, overspend or gradually corrupt their own context, so each transaction is bounded by:

- a **turn budget** $T_{\max}$ (for example $T \leq 5$);
- a **cost budget** $C_{\max}$ in tokens and dollars;
- **optimistic concurrency control**: each record carries a monotonic **version**; updates use PostgreSQL `SELECT ... FOR UPDATE`, and conflicting concurrent writes return HTTP 409.

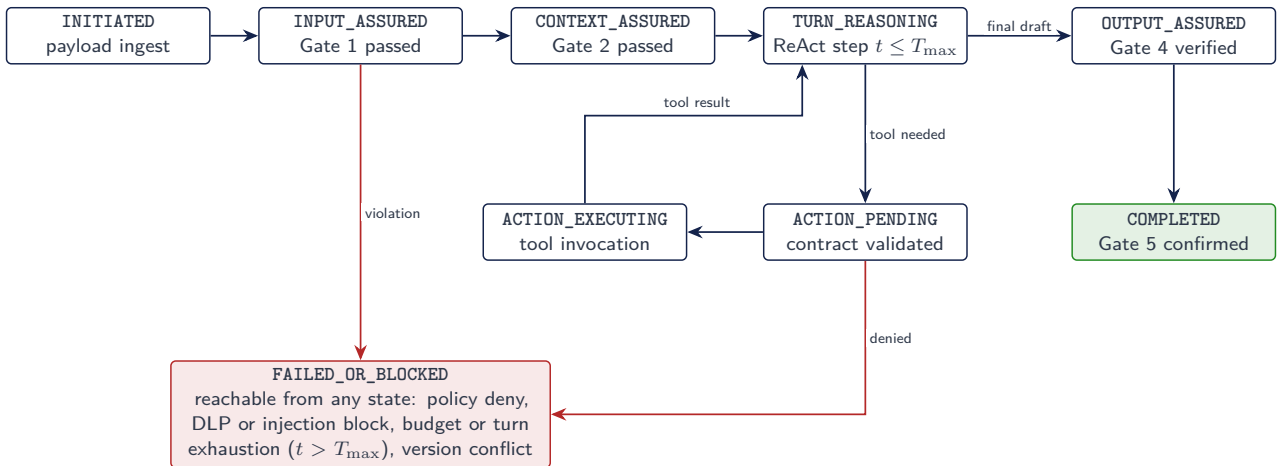


Figure 8: Target AI Transaction state machine (versioned, with the governed ReAct loop).

Implementation status: AI Transaction state machine

The versioned AI Transaction state machine shown in Figure 8 is fully implemented in `services/transaction_manager.py` and backed by PostgreSQL migration 005. Each record carries a strict monotonic **version** column for optimistic concurrency control. Furthermore, state transitions to **COMPLETED** strictly enforce the dual invariant: a verified **OutputAssuranceRecord** and a verified **OutcomeVerificationRecord** must exist for the active transaction, preventing unverified or hallucinated completions from committing.

7.3 The AI Action Contract

Before any consequential operation, the agent must construct an explicit contract:

$$\mathcal{A} = \langle \text{ActionID}, \text{AgentID}, \text{Capability}, \text{Tool}, \text{TargetResource}, \mathbf{P}_{\text{params}}, \Phi_{\text{pre}}, \Psi_{\text{post}}, \mathcal{R}_{\text{comp}} \rangle \quad (1)$$

Φ_{pre} is a machine-checkable precondition (for example, $\text{balance} \geq \$500$); Ψ_{post} is the expected post-condition (row created, status **ACTIVE**); and $\mathcal{R}_{\text{comp}}$ is a compensation strategy in the spirit of sagas [14], executed if the action fails midway.

8 Gate 5: Epistemic Reality Verification

Epistemic discrepancy arises when the model’s text assertions and physical system reality diverge: an agent asserts that an action succeeded when it failed or never ran (phantom success), or reports an error when external state was actually mutated. Gate 5 runs independent, non-model verifier probes against target environments and reconciles declared postconditions with physical ground truth.

8.1 Four observation classes and seven canonical outcome statuses

To prevent conflating an HTTP transport acknowledgement with genuine state mutation, Gate 5 formalises four explicit observation classes:

1. **OBS_DIRECT** (Authoritative): Direct, synchronous inspection of the target ACID database, file system, or infrastructure state.
2. **OBS_EVENTUAL** (Asynchronous): Eventual consistency confirmed via polling queues, webhook delivery, or distributed job IDs.
3. **OBS_INFERRED** (Indirect): Correlated evidence observed through telemetry, side-effect logs, or downstream metrics.
4. **OBS_BLIND** (Unobservable): Write-only sinks (e.g. unmonitored syslog, third-party black-box APIs) with zero readback channel. Confidence is strictly 0.00.

Outcomes are classified into seven canonical statuses (Table 3, Figure 9). A fundamental architectural invariant is enforced: *Tool acknowledgement* $\neq$ *Verified outcome*. Transport acknowledgements cannot be silently promoted to confirmed success.

8.2 Production verifier adapters and monotonic state transitions

The reality verification engine in `services/reality_verifier.py` coordinates four production verifier adapters:

- **DatabaseVerifierAdapter:** Executes read-only parametrized SQL checks. Uses AST and token scanning to reject mutating keywords (**INSERT**, **UPDATE**, **DELETE**, **DROP**) and semicolon-chained multi-statements.
- **RestApiVerifierAdapter:** Executes idempotent GET/HEAD probes against external REST endpoints. Enforces strict anti-SSRF IP/CIDR blocking and re-evaluates all HTTP 301/302/307/308 redirect targets to prevent DNS rebinding or internal pivot attacks.
- **CeleryWorkerVerifierAdapter:** Probes asynchronous job queues, task states, and worker execution results.
- **LogAuditVerifierAdapter:** Inspects tamper-evident structured logs and storage artifacts.

Outcome status	Observation class	Physical reality and invariant
SUCCESS_CONFIRMED	OBS_DIRECT	Postconditions Ψ_{post} verified directly in target system. Transaction committed.
SUCCESS_EVENTUALLY_OBSERVED	OBS_EVENTUAL	Asynchronous worker/queue confirms execution within retry timeout.
ACKNOWLEDGED_UNVERIFIED	OBS_INFERRED	Tool returned HTTP 200/ACK; destination state uninspected. Cannot be released as confirmed.
FAILED	OBS_DIRECT/INFERRED	Execution failed or negative postcondition verified ($\Delta\mathcal{S} = \emptyset$).
PARTIAL	OBS_DIRECT	Multi-action batch partially applied; state inconsistency flagged.
UNKNOWN	OBS_INFERRED	Probe timed out or evidence ambiguous; held in reconciliation queue.
UNOBSERVABLE	OBS_BLIND	Write-only sink; cannot confirm or deny mutation. High-uncertainty tag attached.

Table 3: The seven canonical outcome statuses of Gate 5 Reality Verification.

State transitions follow a **monotonic state machine**: transitions can only advance along paths of increasing empirical evidence (e.g. UNKNOWN $\rightarrow$ SUCCESS_EVENTUALLY_OBSERVED $\rightarrow$ SUCCESS_CONFIRMED). A terminal status (FAILED, SUCCESS_CONFIRMED) is irrevocable.

8.3 Canonical cryptographic fingerprinting: `mirage.outcome.v1`

Every verification record is canonicalized into sorted UTF-8 JSON under schema `mirage.outcome.v1` and cryptographically hashed:

$$H_{\text{outc}} = \text{SHA-256}(\text{"mirage.outcome.v1:"} \parallel \text{canonical_json}(\text{OutcomeRecord})) \quad (2)$$

This fingerprint binds the tenant ID, transaction ID, action contract hash, verifier adapter, observation class, and timestamp, guaranteeing audit non-repudiation.

8.4 Output $\leftrightarrow$ Outcome consistency reconciliation

Even if Gate 4 verifies that candidate text is well-formed, an agent may still output text asserting an unconfirmed real-world action (e.g. *"I have transferred \$500 to account X"*). The Gate 5 interlock function `reconcile_output_with_outcome()` runs regex assertion matchers against the output text. If the text asserts success while the physical outcome is FAILED, UNKNOWN, or ACKNOWLEDGED_UNVERIFIED, Gate 5 blocks release and rewrites the completion to state reality honestly (e.g. *"The transfer request was submitted (ACK), but final confirmation is pending verification"*).

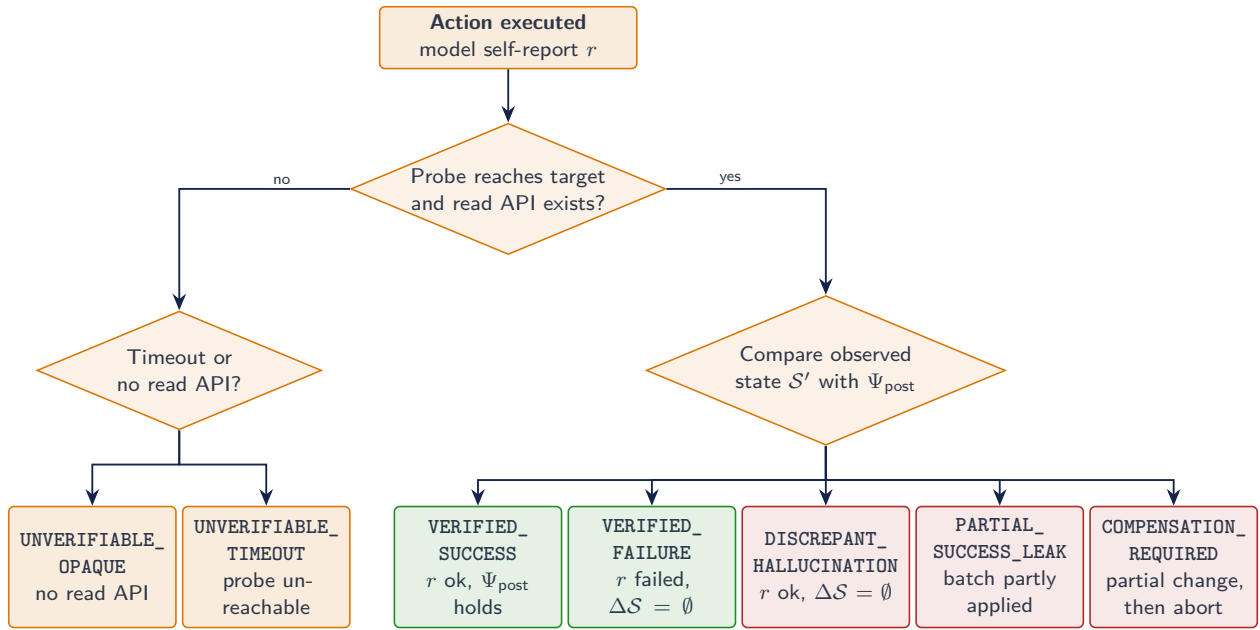


Figure 9: Gate 5 decision tree mapping probe outcomes to the seven epistemic states (green: consistent, amber: unverifiable, red: discrepancy requiring action).

9 Security Architecture and Threat Model

9.1 Resolved P0 identity-boundary vulnerabilities

A pre-implementation audit identified three critical P0 vulnerabilities, all permanently closed (Table 4).

ID	Vulnerability	Risk	Resolution
P0-1	Unauthenticated demo-token issuer	<code>/v1/auth/demo-token</code> let any caller choose <code>tenant_id</code> and <code>role</code> and receive a signed JWT.	Route deleted from gateway and frontend; tokens issued only through authenticated admin channels.
P0-2	Hard-coded super-admin credential	A static key in auth middleware granted global admin rights without database verification.	Credential removed; API keys and JWTs resolved only by stored procedures <code>mirage_resolve_api_credential</code> and <code>mirage_resolve_jwt_identity</code> .
P0-3	Auto-tenant provisioning (IDOR)	Persistence created unknown tenant IDs on write, enabling cross-tenant spoofing.	Replaced by fail-closed <code>SELECT ... FOR UPDATE</code> lookups against pre-registered tenants.

Table 4: Remediated P0 vulnerabilities.

In addition, caller-controlled headers (`X-Role`, `X-Tenant-ID`, `X-Agent-ID`, `X-Capability`) are stripped at the ASGI layer for both HTTP and WebSocket traffic. Figure 10 shows how this combines with PostgreSQL row-level security.

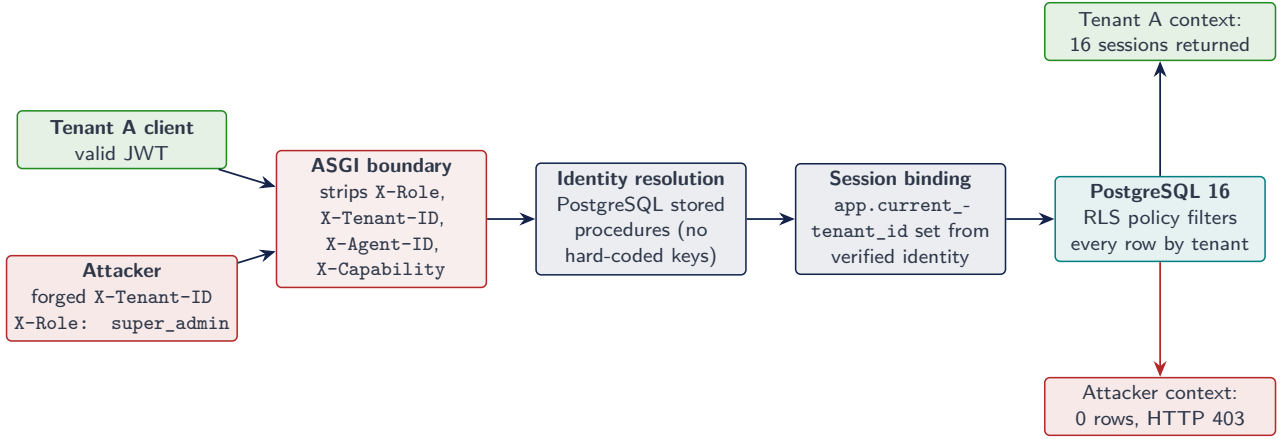


Figure 10: Perimeter defence in depth: header stripping, database-resolved identity and row-level security.

9.2 Dynamic information-flow control

To keep indirect prompt injection from leaking private data, labels form a lattice [12]:

$$\text{PUBLIC} \sqsubseteq \text{INTERNAL} \sqsubseteq \text{CONFIDENTIAL} \sqsubseteq \text{RESTRICTED_PII} \sqsubseteq \text{SECRET_CREDENTIAL} \quad (3)$$

If an agent consumes a chunk labelled ℓ_1 and a tool return labelled ℓ_2 , its working memory inherits the least upper bound $\ell_{\text{state}} = \ell_1 \sqcup \ell_2$. An agent whose state satisfies $\ell_{\text{state}} \sqsupseteq \text{CONFIDENTIAL}$ may not invoke external network tools unless declassification is explicitly approved (Figure 11).

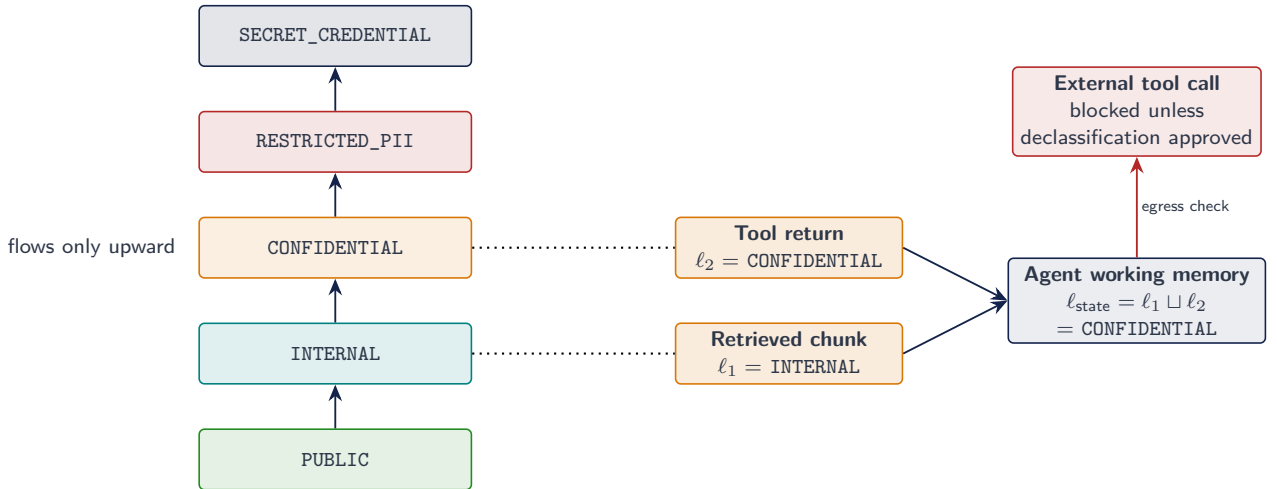


Figure 11: Security lattice and taint propagation: the join of two labels governs egress.

9.3 Sliding-window blast radius

To stop an adversary from splitting a damaging action into many benign-looking ones, Gate 3 tracks decayed cumulative risk over a window W :

$$\mathcal{B}(t) = \sum_{k \in \mathcal{A}(t, W)} \omega(a_k) \text{Impact}(a_k) e^{-\lambda(t-t_k)} \quad (4)$$

where $\mathcal{A}(t, W)$ is the set of actions in the window, $\omega(a_k)$ a per-action-class weight, $\text{Impact}(a_k)$ its financial or operational sensitivity, and $\lambda > 0$ a decay rate. If admitting a new action would push $\mathcal{B}(t)$ above $\mathcal{B}_{\text{max}}$, it is denied regardless of its individual risk (Figure 12; fully implemented in `services/action_governor.py`).

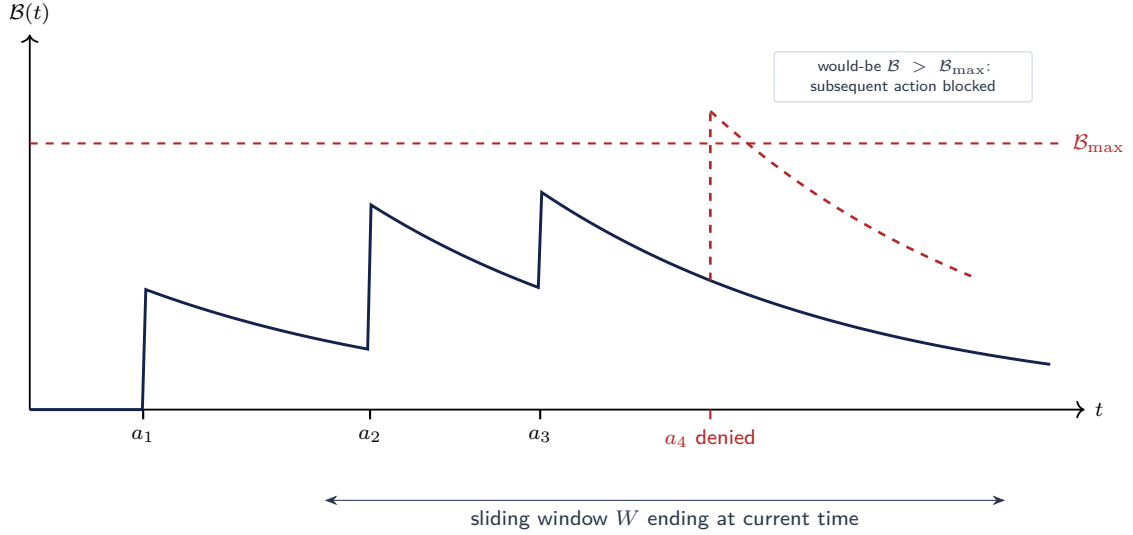


Figure 12: Illustrative blast-radius dynamics ($\lambda = 0.35$, $\omega = 1$, impacts 1.0, 1.2, 0.8, 1.4): the fourth action would breach $B_{\max}$ and is denied.

9.4 Perimeter hardening: GovernedToolProxy, anti-SSRF, and single-use approvals

Production execution across Gates 3 and 5 enforces rigorous physical perimeter isolation:

- **GovernedToolProxy as execution boundary:** The proxy in `services/governed_tool_proxy.py` intercepts all external calls. Direct tool invocation by agents is rejected (`ToolProxyBypassError`); external credentials reside strictly within proxy memory.
- **Anti-SSRF with redirect re-inspection:** HTTP requests and probes filter private IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, AWS metadata 169.254.169.254). Redirects (301/302/307/308) re-evaluate destination addresses, thwarting DNS rebinding and internal network pivots.
- **Context-bound single-use approval tickets:** Human approval tickets are cryptographically bound via HMAC-SHA256 to the transaction ID, actor identity, action contract hash, and a cryptographic nonce. Replay attempts and self-approvals by requesting agents are unconditionally blocked.
- **Read-only SQL verification:** Reality probes inspect target databases using parametrized SELECT queries; SQL parser tokenization rejects all mutating commands (INSERT, UPDATE, DELETE, DROP) and multi-statement semicolons.

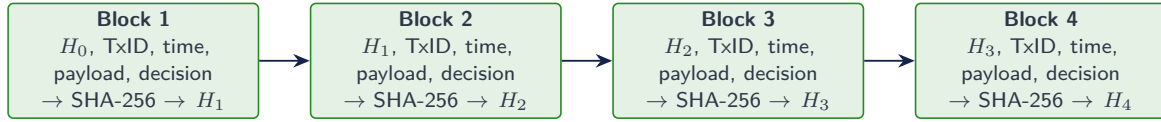
9.5 Tamper-evident audit ledger

Every decision, action and policy evaluation is chained:

$$H_t = \text{SHA-256}(H_{t-1} \parallel \text{TransactionID} \parallel \text{Timestamp} \parallel \text{ActionPayload} \parallel \text{Decision}) \quad (5)$$

Changing any historical record invalidates the hashes of all later records, supporting forensic non-repudiation for SOC 2, HIPAA and EU AI Act audits (Figure 13).

Intact chain: every stored hash matches the recomputed hash



Tampered chain: one altered record breaks every later link

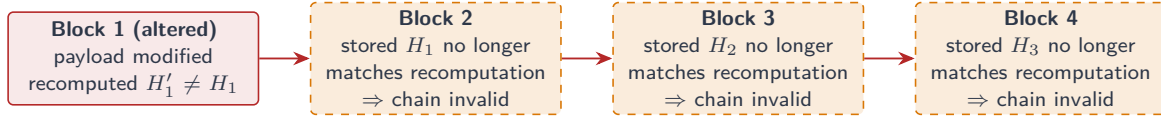


Figure 13: SHA-256 hash-chained audit ledger: intact versus tampered chain.

10 Gate 4: Output Assurance and the Verification Engine

10.1 Evidence signals

The MIRAGE 2.x scientific pipeline is the algorithmic core of Gate 4. Compound text is split into atomic claims, each verified through orthogonal signals (Table 5, Figure 14).

Signal	Technology	Measured latency	Failure mitigated
RAV – dense retrieval	Qdrant HNSW + MPNet-768 [9]	~45 ms	Fabrications unsupported by the reference corpus.
SCS – consistency sampling	Parallel Groq sampling, $T = 0.7$	~800 ms (<2 ms on cache hit)	Semantic dispersion under model uncertainty.
NLI – entailment	DeBERTa-v3 cross-encoder [8]	~80 ms	Contradiction against verified references.
VGS – visual grounding	OpenCLIP + LLaVA-1.6	~30 ms / ~900 ms	Fabricated visual objects.
ICS – internal consistency	Semantic chunk consistency	~25 ms	Self-contradiction across paragraphs.

Table 5: Multi-signal evidence matrix of the Gate 4 verification engine.

10.2 HRS fusion and the dominance rule

Each signal is normalised so that higher means riskier. The claim-level Hallucination Risk Score is

$$\text{HRS}(c_i) = w_1 \text{RAV}(c_i) + w_2 \text{SCS}(c_i) + w_3 \text{NLI}(c_i) + w_4 \text{ICS}(c_i), \quad (6)$$

with canonical weights $w_1 = 0.35$, $w_2 = 0.30$, $w_3 = 0.25$, $w_4 = 0.10$ (summing to one). Because a response containing one fatal falsehood is not safe merely because the rest is accurate, the **critical-hallucination dominance rule** is applied, where $w_{\text{crit}}(c_i)$ is the criticality weight of claim c_i :

$$\text{HRS}_{\text{resp}} = \max \left(\frac{\sum_{i=1}^K w_{\text{crit}}(c_i) \text{HRS}(c_i)}{\sum_{i=1}^K w_{\text{crit}}(c_i)}, 0.80 \cdot \max_{1 \leq i \leq K} \text{HRS}(c_i) \right) \quad (7)$$

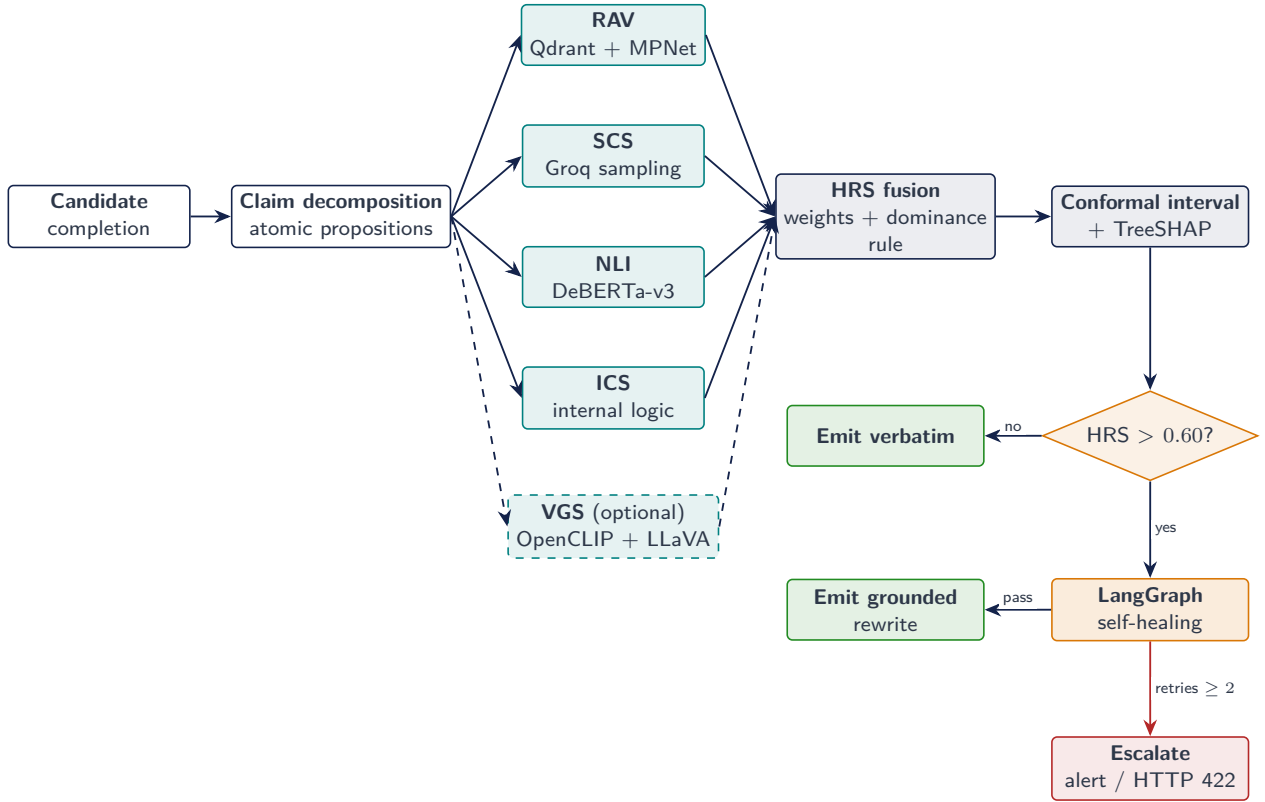


Figure 14: Gate 4 verification engine: from claim decomposition to emission, self-healing or escalation.

10.3 Split-conformal calibration and attribution

Prediction intervals are calibrated on held-out non-conformity scores $\{s_i\}_{i=1}^n$ [3, 4, 5]:

$$\hat{q} = \text{Quantile}\left(\{s_i\}_{i=1}^n, \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right), \quad \mathcal{C}(\mathbf{x}) = [\max(0, \text{HRS} - \hat{q}), \min(1, \text{HRS} + \hat{q})] \quad (8)$$

At $1 - \alpha = 0.95$ and $n = 1,000$ calibration examples, the measured $\hat{q} = 0.0650$ gives an interval width of $0.130 < 0.20$. The coverage guarantee is *marginal* and assumes calibration and deployment data are exchangeable; the Mondrian variant provides coverage per category. TreeSHAP decomposes the score additively, $\text{HRS}(\mathbf{x}) = \phi_0 + \sum_{j=1}^M \phi_j(\mathbf{x})$ [6, 7].

10.4 LangGraph closed-loop self-healing

When $\text{HRS} > 0.60$, a compiled LangGraph state machine isolates contradicted claims, retrieves targeted Qdrant evidence, performs a grounded rewrite and applies a mandatory re-verification gate $\text{HRS}_{\text{rewrite}} \leq 0.30$ before emission (Figure 15).

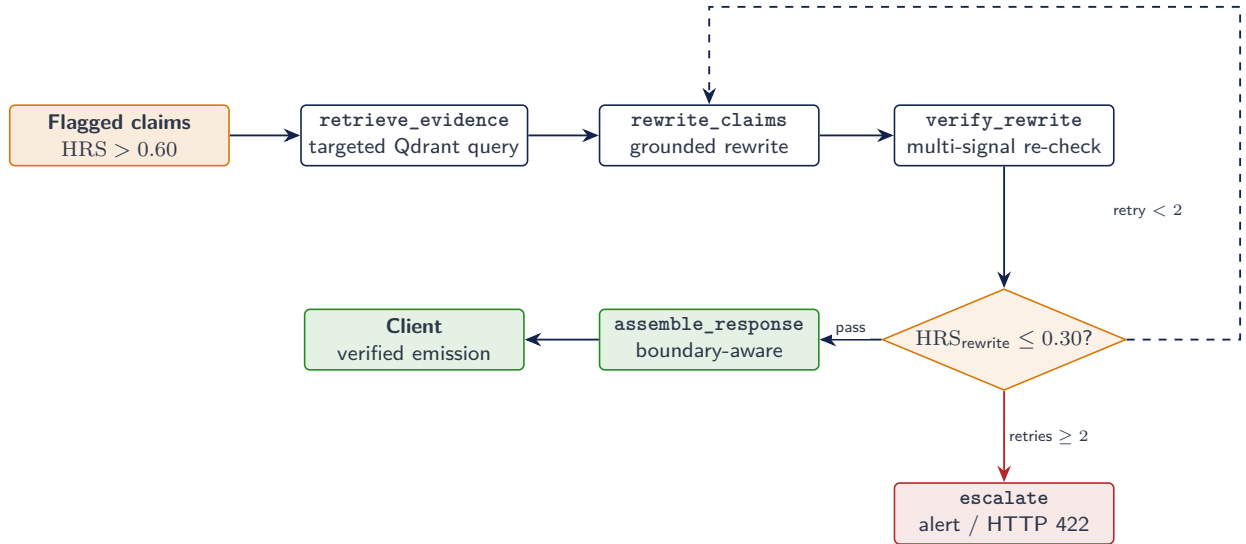


Figure 15: LangGraph closed-loop self-correction with a mandatory re-verification gate.

11 Validated Empirical Telemetry

11.1 Runtime model identity probes

Direct probes of the Groq inference endpoint (api.groq.com/openai/v1/chat/completions) established production telemetry:

- **Active production model:** allam-2-7b (system fingerprint fp_ff744b2c7a).
- **Measured latency:** generation 15.18 ms, queue time 46.18 ms.
- **Upstream deprecation:** llama-3.1-70b-versatile and llama-3.1-8b-instant were decommissioned (HTTP 400); all harnesses were aligned to allam-2-7b.

11.2 Grounded versus hallucinated verification

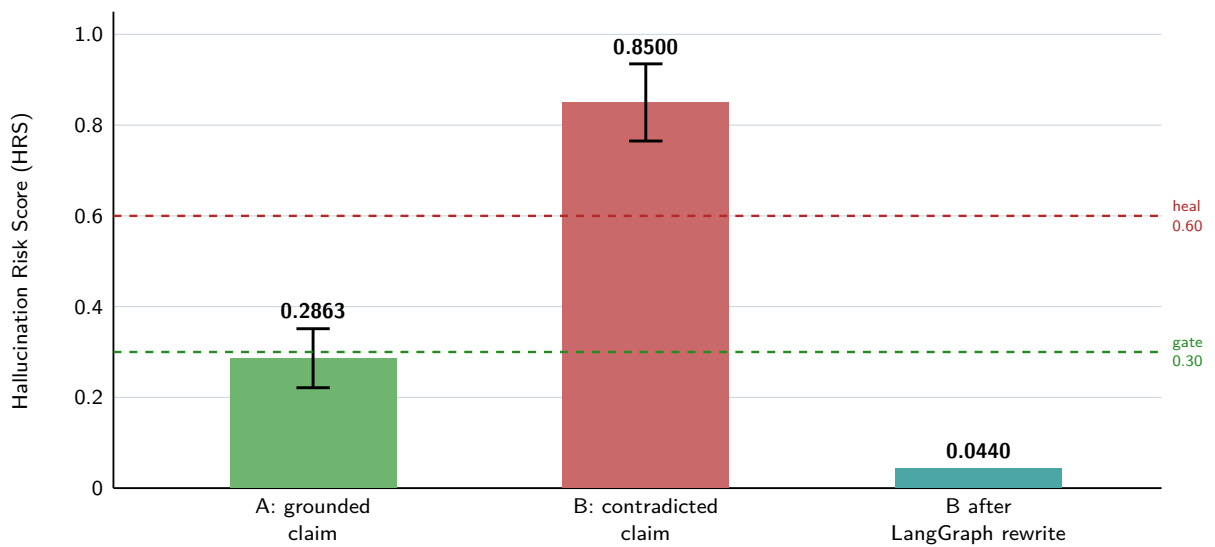


Figure 16: HRS for the two live scenarios with 95% conformal intervals ($\hat{q} = 0.0650$). The rewrite drops scenario B from 0.85 to 0.044, below the 0.30 re-verification gate.

Metric	Scenario A: grounded	Scenario B: contradicted
Prompt	“Who discovered penicillin and when?”	“Who discovered penicillin and where?”
Candidate	“Alexander Fleming discovered penicillin in 1928 at St. Mary’s Hospital in London.”	“Alexander Fleming discovered penicillin in 1999 at Harvard University in Boston.”
Qdrant match	Chunk doc_a745..., similarity 0.8164	Chunk doc_a745..., similarity 0.7912
Claims	c_1 Fleming 1928: SUPPORTED (0.190); c_2 St. Mary’s: NEUTRAL (0.446)	c_1 Fleming 1999 Harvard: CONTRADICTED (0.850)
Overall HRS	0.2863 (low/medium)	0.8500 (critical)
Conformal interval	[0.2213, 0.3513]	[0.7650, 0.9350]
Attribution (TreeSHAP, as reported)	SCS 0.570, RAV 0.266, NLI 0.123	NLI 0.520, SCS 0.320, RAV 0.160
LangGraph	Bypassed (≤ 0.60)	Triggered (> 0.60)
Rewritten output	Original preserved verbatim	“Alexander Fleming discovered penicillin in 1928 at St. Mary’s Hospital in London.”
Re-verification	N/A	HRS _{rewrite} = 0.0440 (gate ≤ 0.30 passed)
Emitted status	Direct emission	Grounded rewrite emitted

Table 6: Live telemetry captured on two production scenarios.

11.3 Persistence and fault-injection probes

- **Row-level security probe.** With `app.current_tenant_id` set to `'tenant_attacker'`, queries returned 0 rows; the authorised tenant context returned all 16 sessions; cross-tenant access returned HTTP 403.
- **Circuit-breaker fault injection.** Port failures injected into Qdrant tripped PyBreaker to OPEN within three failures; the fallback sparsity penalty applied without any unhandled HTTP 500.

11.4 Automated test regression suite telemetry

The MIRAGE 3.0 implementation across Phases 0 through 5.5 was subjected to comprehensive test automation and static verification:

- **Pytest regression suite:** 511 automated tests passed (0 failures, 0 errors, 59 warnings). All 59 warnings were audited (57 datetime UTC deprecations, 1 Pydantic root-validator deprecation, 1 Starlette lifespan warning) and confirmed non-functional.
- **Linters compliance:** `ruff check` . passed with 0 errors across all Python modules.
- **Type safety:** `mypy -strict` passed with 0 errors across all hardened Phase 1–5.5 service implementations (`services/`, `gateway/`, `shared/`, `hrs_engine/`).
- **Frontend compilation:** React 18 / Vite production build (`npm run build`) transformed 1,603 modules in 5.86s with zero warnings, strictly adhering to the JavaScript/JSX-only constraint.
- **Live demonstration:** `scripts/demo_mirage3_execution_assurance.py` passed all nine end-to-end scenarios.

12 Cost Optimisation: The Verification Escalation Ladder

Assurance must not multiply cost and latency. MIRAGE handles each case at the cheapest rung that can resolve it (Figure 17); expensive rungs run only when cheaper ones are inconclusive or policy requires them.

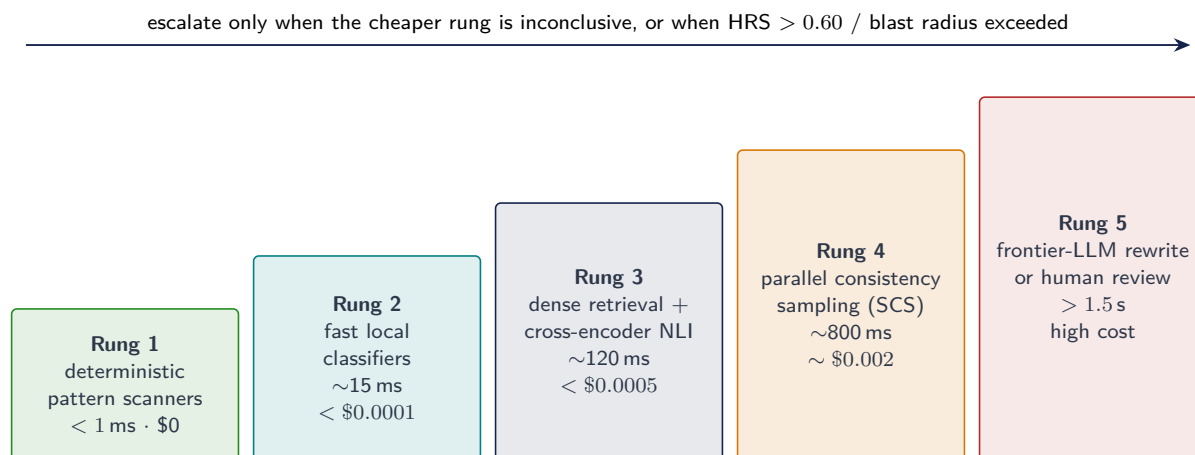


Figure 17: Verification escalation ladder: cost and latency grow with each rung.

13 Comprehensive Competitive Landscape

MIRAGE 3.0 is compared with nine commercial and academic systems. The table is split into two parts so every column stays legible; cell entries reflect each tool’s public documentation at the time of writing and should be re-verified before external publication.

Dimension	MIRAGE 3.0	SelfCheckGPT	FactScore	Guardrails AI	NeMo Guardrails
Category	Execution fabric	Offline algorithm	Offline evaluation	Python SDK	Dialog framework (Colang)
Assurance scope	Five gates	Output only	Output only	Input / output	Input / output
Action governance	Capability ABAC	None	None	None	Programmable
Reality verification	Seven-state probes	None	None	None	None
Dynamic IFC	Taint tracking	None	None	None	None
Multi-signal	4+1 orthogonal	SCS only	RAG + NLI	Regex / LLM validators	Vector embeddings
Uncertainty	Split-conformal	None	None	None	None
Self-correction	LangGraph gate	None	None	Exception handling	Flow redirect
Multi-tenancy	Postgres RLS	None	None	In-memory	None
Audit ledger	SHA-256 chain	None	None	Plain logs	Plain logs
Deployment	Sovereign VPC	Script	Script	Microservice	Container

Table 7: Architectural comparison, part A: academic baselines and guardrail libraries.

Figure 18 visualises the lifecycle coverage implied by the “assurance scope” and “action governance” rows.

Dimension	MIRAGE 3.0	Cleanlab TLM	Galileo	TruLens	Langfuse	Bedrock Guardrails
Category	Execution fabric	Scoring API	SaaS platform	SDK evaluation	Tracing SaaS	Cloud proxy
Assurance scope	Five gates	Output only	Output only	Output only	Observability	Input / output filters
Action governance	Capability ABAC	None	None	None	None	None
Reality verification	Seven-state probes	None	None	None	None	None
Dynamic IFC	Taint tracking	None	None	None	None	None
Multi-signal	4+1 orthogonal	Perplexity-based	LLM-as-judge	RAG triad	Metadata	Regex / keywords
Uncertainty	Split-conformal	Heuristic	Heuristic	None	None	None
Self-correction	LangGraph gate	Best-of-N	None	None	None	Drop / mask
Multi-tenancy	Postgres RLS	Cloud tenant	SaaS tenant	SaaS tenant	SaaS tenant	AWS IAM
Audit ledger	SHA-256 chain	Plain logs	Plain logs	OpenTelemetry	ClickHouse	CloudWatch
Deployment	Sovereign VPC	Proprietary	Proprietary	Open / cloud	Open / cloud	AWS only

Table 8: Architectural comparison, part B: commercial evaluation and cloud platforms.

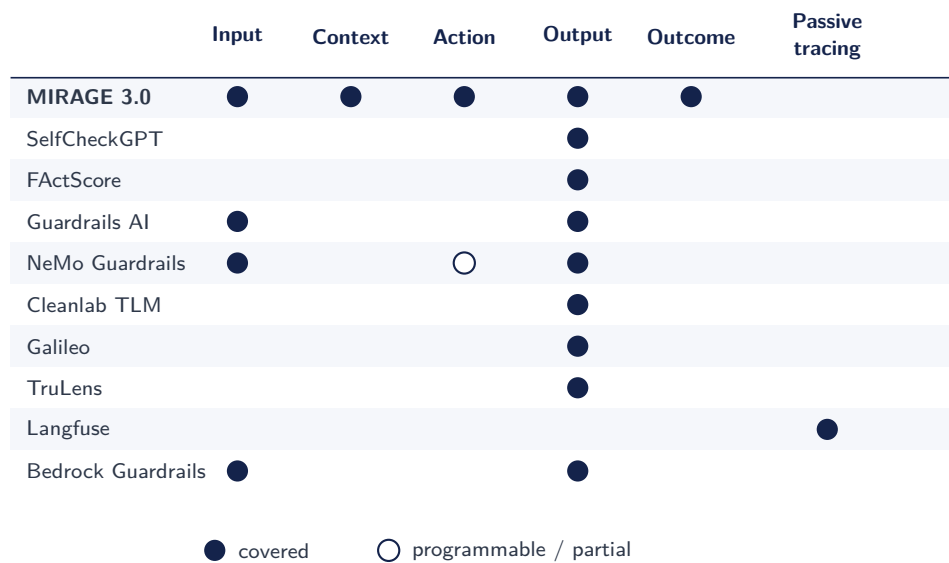


Figure 18: Lifecycle coverage of compared systems, derived from the “assurance scope” and “action governance” rows.

13.1 Architectural differentiation

- **Versus guardrail libraries (Guardrails AI, NeMo).** They rely on procedural validators or dialog flows; neither provides calibrated probabilistic risk scores or database-enforced multi-tenancy.
- **Versus observability platforms (Langfuse, TruLens).** They record traces after the fact; they cannot block an unauthorised action inline, enforce capabilities or verify outcomes.
- **Versus offline evaluators (SelfCheckGPT [1], FActScore [2]).** These are research algorithms, not sub-second inline mediators, and offer no self-correction loop.

14 Implementation Status Inventory

In the interest of academic integrity, Table 9 and Figure 19 separate what is running and tested from what remains scheduled on the research roadmap.

Component	Status	Evidence and source artefacts
FastAPI gateway core	Implemented	Reverse proxy, route decorators, token-bucket limiter in <code>gateway/</code> .
Perimeter security and auth	Implemented	P0 issues closed; stored procedures, ASGI header stripping; <code>tests/security/</code> .
Gate 1: deterministic input	Implemented	DLP redaction, injection indicators, dynamic risk escalation in <code>services/input_assurance.py</code> .
Gate 2: context assurance	Implemented	Provenance hashing, freshness decay, lattice taint tracking in <code>services/context_assurance.py</code> .
Gate 3: action governor	Implemented	GovernedToolProxy, cryptographic contracts, blast radius, single-use tickets in <code>services/action_governor.py</code> .
Gate 4: multi-signal engine	Implemented	RAV (Qdrant), SCS (Groq), NLI (DeBERTa-v3), HRS, conformal bounds, completion invariants in <code>services/output_assurance.py</code> .
LangGraph self-correction	Implemented	Closed-loop state machine with re-verification gate (≤ 0.30) in <code>correction_agent/orchestrator.py</code> .
Gate 5: reality verifier	Implemented	4 observation classes, 7 canonical statuses, 4 adapters, Output $\leftrightarrow$ Outcome reconciliation in <code>services/reality_verifier.py</code> .
AI Transaction state machine	Implemented	Monotonic states, optimistic versioning (<code>version</code>), completion invariants in <code>services/transaction_manager.py</code> .
Persistence and audit	Implemented	PostgreSQL 16 RLS, unbroken SHA-256 ledger, MongoDB traces verified in live test suite.
React 18 dashboard	Implemented	Vite build (JS/JSX only): 1,603 modules transformed in 5.86 s with zero warnings.
Tier 2 academic corpora run	Roadmap	$N \geq 1,000$ HaluEval / TruthfulQA ablation run with paired bootstrap, planned for final thesis defence.

Table 9: Implementation status inventory across platform subsystems.

Implemented (511 passing tests)
 Roadmap (thesis defence)

Assurance gates (all implemented)



Platform components (all implemented)



Figure 19: Implementation status by gate and platform component: all five assurance gates fully implemented and validated (11 implemented, 1 roadmap item).

15 Live Demonstration Walkthrough

The demonstration script

```
scripts/demo_mirage3_execution_assurance.py
```

executes nine sequential end-to-end scenarios validating the entire assurance pipeline:

1. **Authoritative identity binding.** JWT issuance via `mirage_resolve_jwt_identity`; tenant and role claims are immutably bound.
2. **Perimeter defence.** Forged headers (`X-Role: super_admin`, `X-Tenant-ID: tenant_victim`) are stripped at the ASGI boundary, and the legacy `/v1/auth/demo-token` route returns HTTP 404.
3. **Gate 1 input assurance.** *Prompt A (clean):* baseline risk 0/100. *Prompt B (SSN and API key):* DLP detects payload, tags `TAINT_CONFIDENTIAL`, redacts secrets. *Prompt C (“IGNORE PREVIOUS SYSTEM INSTRUCTIONS”):* immediate `BLOCK` with zero downstream inference or tool cost.
4. **Gate 2 context assurance and delimiter containment.** Delimiter escaping, RAG provenance verification, freshness decay, and security lattice taint propagation ($\ell_{\text{state}} = \ell_1 \sqcup \ell_2$).
5. **Gate 3 action assurance and GovernedToolProxy.** Capability checks, schema validation, parameter hash locking, sliding blast radius, and context-bound approval tickets. Direct tool bypass is physically blocked.
6. **AI Transaction lifecycle and ReAct turn budgets.** Transaction envelope with a bounded 3-turn ReAct budget advances through versioned states with optimistic locking (`SELECT ... FOR UPDATE`) and rollback on failure.
7. **Gate 4 output assurance and self-healing.** Grounded Fleming-1928 claim ($\text{HRS} = 0.185$) emitted directly; hallucinated Harvard-1999 claim ($\text{HRS} = 0.850$) activates LangGraph self-healing, rewriting from Qdrant evidence to $\text{HRS}_{\text{rewrite}} = 0.044 \leq 0.30$.
8. **Cryptographic SHA-256 audit ledger.** Immutable hash-chained ledger; altering 1 bit in Block 1 invalidates Block 2 and all subsequent records (Figure 13).
9. **Gate 5 reality verification and outcome assurance.**
 - **9A (OBS_DIRECT):** Synchronous ACID database inspection confirming postcondition rows (100% confidence).

- **9B (OBS_INFERRED):** Distinguishes transport HTTP 200 acknowledgement from verified destination reality (ACKNOWLEDGED_UNVERIFIED).
- **9C (Output ↔ Outcome interlock):** `reconcile_output_with_outcome()` catches agent text falsely asserting unconfirmed success and rewrites it to state reality honestly.
- **9D (OBS_BLIND):** Write-only sinks correctly qualified as UNOBSERVABLE with 0.00 confidence.

16 Research Innovations versus Inherited Heritage

Dimension	Heritage (MIRAGE 2.x)	Novel in MIRAGE 3.0
Assurance scope	Output-text hallucination detection only.	Five assurance gates across input, context, action, output and outcome.
Governed primitive	Stateless (prompt, completion) checks.	AI Transaction envelope with bounded ReAct turns and budgets.
Action and tool safety	Unmanaged tool calls.	AI Action Contracts , capability-based security, blast-radius bounds.
Reality verification	None; relied on model output.	Epistemic Reality Verification over seven states.
Data-flow security	Static RBAC in the database.	Dynamic information-flow control with lattice taint tracking.
Statistical guarantees	Mondrian split-conformal intervals at $1 - \alpha = 0.95$.	Preserved and integrated into Gate 4 decision boundaries.
Self-correction	LangGraph state machine.	Preserved; augmented with boundary-aware token replacement and safety re-checks.

Table 10: Inherited heritage versus novel contributions.

17 Strengths, Limitations and Threats to Validity

17.1 Strengths

1. **Assurance beyond text:** blocks dangerous actions, context poisoning and phantom successes that text guardrails miss.
2. **Distribution-free uncertainty:** split-conformal intervals with finite-sample marginal coverage rather than heuristic scores.
3. **Hardened multi-tenancy:** engine-level PostgreSQL RLS plus ASGI header stripping removes the audited IDOR paths.
4. **Adaptive cost:** the escalation ladder runs cheap checks first.

17.2 Limitations

1. **Sampling latency.** Uncached SCS adds 800–1,800 ms (mitigated by Redis caching and a bypass for latency-critical tenants).
2. **Corpus dependency.** RAV needs an ingested reference corpus; un-indexed domains incur the 0.85 sparsity risk penalty.
3. **Multimodal hardware.** Full LLaVA-1.6 grounding needs a high-memory GPU (for example NVIDIA A10G, 24 GB); developer machines degrade to text-only verification.
4. **Cold start.** Loading SentenceTransformers and DeBERTa cross-encoders takes 25–40 s at container launch.

17.3 Threats to validity

- Calibration used $n = 1,000$ examples; coverage is marginal and relies on exchangeability with deployment data.
- Telemetry in Table 6 covers production demonstration scenarios; full large-scale Tier 2 empirical evaluation ($N \geq 1,000$) remains scheduled for final thesis defence.
- While all five gates are implemented and validated across 511 unit/integration tests, high-concurrency distributed load testing under peak multi-tenant traffic remains an active engineering item.

18 Phased Roadmap and Future Work

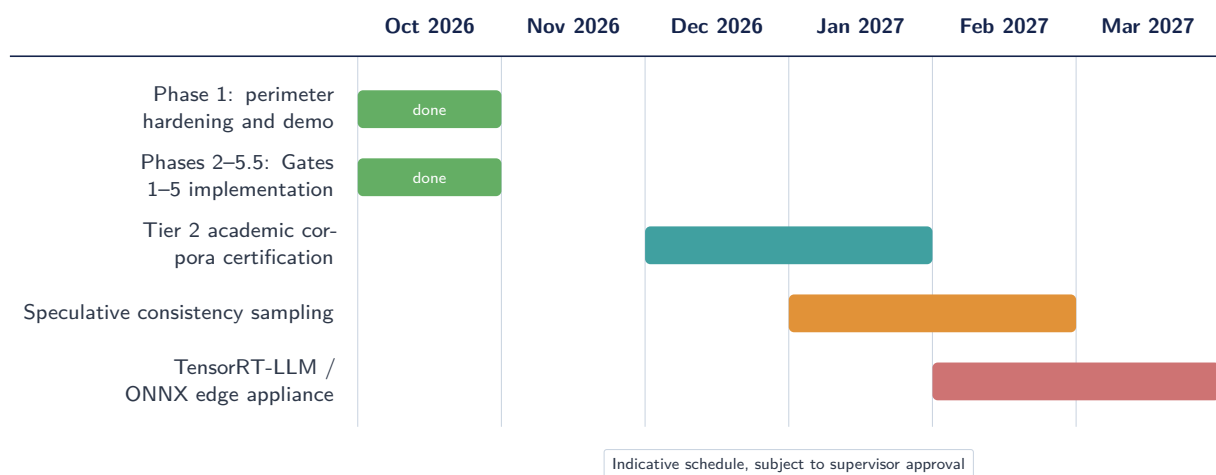


Figure 20: Phased engineering and research roadmap: core five-gate architecture completed; empirical corpora certification scheduled.

1. **Phases 0–5.5: five-gate architecture and hardening (Completed).** Full implementation and validation of Gates 1–5, GovernedToolProxy, monotonic reality probes, anti-SSRF redirect protections, and Output $\leftrightarrow$ Outcome reconciliation (511 automated regression tests passing).
2. **Tier 2 academic certification.** Full-corpus evaluation on HaluEval [15] ($N = 10,000$), TruthfulQA [16] ($N = 817$) and FActScore [2] ($N = 183$), with paired bootstrap tests ($B = 10,000$) and an 11-way Bonferroni correction.
3. **Speculative consistency sampling.** Lightweight 1B draft models for SCS consensus; the projected goal is a substantial reduction in Groq token cost (target, not yet measured).

4. **Edge quantisation.** Compile DeBERTa-v3 and FLAN-T5 to FP8/INT4 ONNX binaries for sovereign on-premise deployment.

19 Conclusion and Faculty Approval Request

Autonomous, tool-executing agents demand a safety architecture that goes beyond output hallucination detection. MIRAGE 3.0 contributes an **AI Execution Assurance Platform** built on five gates, governed AI transactions, capability-based security, and epistemic reality verification. With all P0 identity vulnerabilities closed, all five assurance gates operational and validated by 511 automated regression tests, a clean React 18 production build, and an end-to-end nine-scenario live demonstration, the project combines rigorous scientific novelty with production-grade systems engineering.

We respectfully submit this report to the faculty examination committee for formal review and approval.

Vedant Panchal (23AIML042)

Student Investigator, B.Tech AI & ML
CSPIT, CHARUSAT

Dax Virani (23AIML076)

Student Investigator, B.Tech AI & ML
CSPIT, CHARUSAT

Project Guide / Supervisor

Department of AI & ML
CSPIT, CHARUSAT

Head of Department (HOD)

Department of AI & ML
CSPIT, CHARUSAT

References

- [1] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. *EMNLP 2023*.
- [2] Min, S., et al. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. *EMNLP 2023*.
- [3] Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.
- [4] Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., & Wasserman, L. (2018). Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523).
- [5] Angelopoulos, A. N., & Bates, S. (2021). A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv:2107.07511.
- [6] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS 2017*.
- [7] Lundberg, S. M., et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56–67.
- [8] He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. arXiv:2111.09543.
- [9] Song, K., Tan, X., Qin, T., Lu, J., & Liu, T.-Y. (2020). MPNet: Masked and permuted pre-training for language understanding. *NeurIPS 2020*.
- [10] Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4), 824–836.
- [11] Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*.
- [12] Denning, D. E. (1976). A lattice model of secure information flow. *Communications of the ACM*, 19(5), 236–243.
- [13] Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308.
- [14] Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD 1987*.
- [15] Li, J., Cheng, X., Zhao, W. X., Nie, J.-Y., & Wen, J.-R. (2023). HaluEval: A large-scale hallucination evaluation benchmark for large language models. *EMNLP 2023*.
- [16] Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA: Measuring how models mimic human falsehoods. *ACL 2022*.
- [17] Anthropic. (2024). Model Context Protocol specification. <https://modelcontextprotocol.io>.
- [18] LangChain. LangGraph documentation. <https://langchain-ai.github.io/langgraph/>.